```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
```

```
df = sns.load_dataset("titanic")
df.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who
0	0	3	male	22.0	1	0	7.2500	S	Third	man
1	1	1	female	38.0	1	0	71.2833	C	First	woman
2	1	3	female	26.0	0	0	7.9250	S	Third	woman
3	1	1	female	35.0	1	0	53.1000	S	First	woman
4	0	3	male	35.0	0	0	8.0500	S	Third	man

Next steps: Generate code with df | New interactive sheet

```
df = df[['age', 'fare', 'survived']]
```

```
df.head()
```

	age	fare	survived
0	22.0	7.2500	0
1	38.0	71.2833	1
2	26.0	7.9250	1
3	35.0	53.1000	1
4	35.0	8.0500	0

Next steps: Generate code with df | New interactive sheet

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import KBinsDiscretizer
from sklearn.model_selection import cross_val_score
from sklearn.compose import ColumnTransformer
```

```
df.dropna(inplace=True)
df.isnull().sum()
```

	0
age	0
fare	0
survived	0

dtype: int64

```
df.head()
```

	age	fare	survived
0	22.0	7.2500	0
1	38.0	71.2833	1
2	26.0	7.9250	1
3	35.0	53.1000	1
4	35.0	8.0500	0

Next steps: Generate code with df | New interactive sheet

```
X = df.iloc[:,0:2]
Y = df.iloc[:,-1]
```

```
X.head()
```

	age	fare
0	22.0	7.2500
1	38.0	71.2833
2	26.0	7.9250
3	35.0	53.1000
4	35.0	8.0500

Next steps: Generate code with X | New interactive sheet

```
Y.head()
```

	survived
0	0
1	1
2	1
3	1
4	0

dtype: int64

```
X_train,X_test,Y_train,Y_test = train_test_split(X,Y,test_size=0.4,random_state
```

```
clf = DecisionTreeClassifier()
clf.fit(X_train,Y_train)
```

▾ DecisionTreeClassifier

```
DecisionTreeClassifier()
```

```
y_pred = clf.predict(X_test)
```

```
accuracy_score(Y_test,y_pred)
```

```
0.5524475524475524
```

```
np.mean(cross_val_score(DecisionTreeClassifier(),X,Y,cv=5,scoring="accuracy"))
```

```
np.float64(0.6176499556781246)
```

```
kb_age = KBinsDiscretizer(n_bins=15,encode="ordinal",strategy="quantile")
kb_fire = KBinsDiscretizer(n_bins=10,encode="ordinal",strategy="quantile")
```

```
trf = ColumnTransformer(
    [
        ("first",kb_age,[0]),
        ("second",kb_fire,[1])
    ]
)
```

```
X_train_trf = trf.fit_transform(X_train)
X_test_trf = trf.fit_transform(X_test)
```

```
trf.named_transformers_["first"].n_bins_
```

```
array([15])
```

```
trf.named_transformers_["second"].n_bins_
```

```
array([10])
```

```
trf.named_transformers_["first"].bin_edges_
```

```
array([array([ 0.42,  6.  , 16.  , 18.  , 20.  , 22.  , 24.  , 26.  , 28.  ,
              30.  , 32.  , 35.  , 38.  , 44.  , 52.  , 71.  ])            ],
      dtype=object)
```

```
trf.named_transformers_["second"].bin_edges_
```

```
array([array([  0.     ,   7.69585,   7.925  ,   9.28335,  12.2875 ,  13.92915,
               21.     ,  27.7354 ,  35.5    ,  72.25   , 512.3292 ])
],
      dtype=object)
```

```
df1 = pd.DataFrame({
    "age" : X_train["age"],
    "age_trf" : X_train_trf[:,0],
    "fare"   : X_train["fare"],
    "fare_trf" : X_train_trf[:,1]
})
```

```
df1.head()
```

	age	age_trf	fare	fare_trf
146	27.0	6.0	7.7958	1.0
155	51.0	13.0	61.3792	8.0
735	28.5	7.0	16.1000	5.0
294	24.0	5.0	7.8958	2.0
0	22.0	4.0	7.2500	0.0

Next steps: Generate code with df1 New interactive sheet

```
df1["age_label"] = pd.cut(x=X_train["age"],bins=trf.named_transformers_["first"
df1["fare_label"] = pd.cut(x=X_train["fare"],bins=trf.named_transformers_["secc
```

```
df1.head()
```

	age	age_trf	fare	fare_trf	age_label	fare_label
146	27.0	6.0	7.7958	1.0	(26.0, 28.0]	(7.696, 7.925]
155	51.0	13.0	61.3792	8.0	(44.0, 52.0]	(35.5, 72.25]
735	28.5	7.0	16.1000	5.0	(28.0, 30.0]	(13.929, 21.0]
294	24.0	5.0	7.8958	2.0	(22.0, 24.0]	(7.696, 7.925]
0	22.0	4.0	7.2500	0.0	(20.0, 22.0]	(0.0, 7.696]

Next steps: Generate code with df1 New interactive sheet

```
clf = DecisionTreeClassifier()
clf.fit(X_train_trf,Y_train)
```

```
▾ DecisionTreeClassifier
DecisionTreeClassifier()
```

```
y_pred2 = clf.predict(X_test_trf)
y_pred2
```

```
array([0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1,
       0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,
       0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1,
       1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0,
       0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0,
       1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0,
       0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0,
       1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1,
       1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0])
```

```
accuracy_score(Y_test,y_pred2)
```

```
0.6293706293706294
```

```
trf_test = trf.fit_transform(X)
```

```
np.mean(cross_val_score(DecisionTreeClassifier(),X,Y,cv=5,scoring="accuracy"))
```

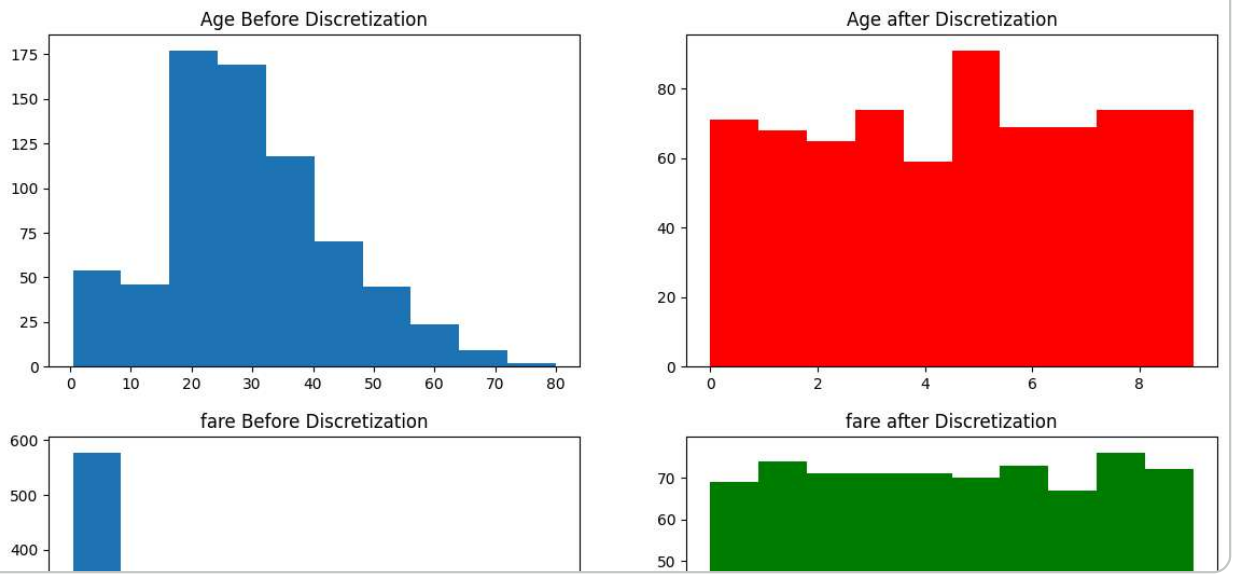
```
np.float64(0.6190485570767261)
```

```
def discretized(bins, strategy, X_train, X_test, Y_tr
    # Create discretizers
    kb_age = KBinsDiscretizer(n_bins=bins, encode="or
    kb_fire = KBinsDiscretizer(n_bins=bins, encode="o

    # ColumnTransformer
    trf = ColumnTransformer([
        ("age_bin", kb_age, [0]),
        ("fire_bin", kb_fire, [1])
    ])

    # Transform training and testing sets
    X_train_trf = trf.fit_transform(X_train)
    X_test_trf = trf.transform(X_test)

    # Train classifier
    clf = DecisionTreeClassifier()
    clf.fit(X_train_trf, Y_train)

    # Predict
    y_pred2 = clf.predict(X_test_trf)
    test_acc = accuracy_score(Y_test, y_pred2)
    print("Test Accuracy:", test_acc)

    # Cross-validation on full transformed data
    X_trf_full = trf.fit_transform(np.vstack((X_train
```

```
    Y_full = np.hstack((Y_train, Y_test))
    cv_acc = np.mean(cross_val_score(DecisionTreeClas
    print("Cross-validation Accuracy:", cv_acc)


    plt.figure(figsize=(14,4))
    plt.subplot(121)
    plt.hist(X["age"])
    plt.title("Age Before Discretization")

    plt.subplot(122)
    plt.hist( X_trf_full[:,0],color = "red")
    plt.title("Age after Discretization")


    plt.figure(figsize=(14,4))
    plt.subplot(121)
    plt.hist(X["fare"])
    plt.title("fare Before Discretization")

    plt.subplot(122)
    plt.hist( X_trf_full[:,1],color = "g")
    plt.title("fare after Discretization")

    return clf, test_acc, cv_acc
```

```
clf, test_acc, cv_acc = discretized(10, "quantile", X_train, X_test, Y_train, Y
```

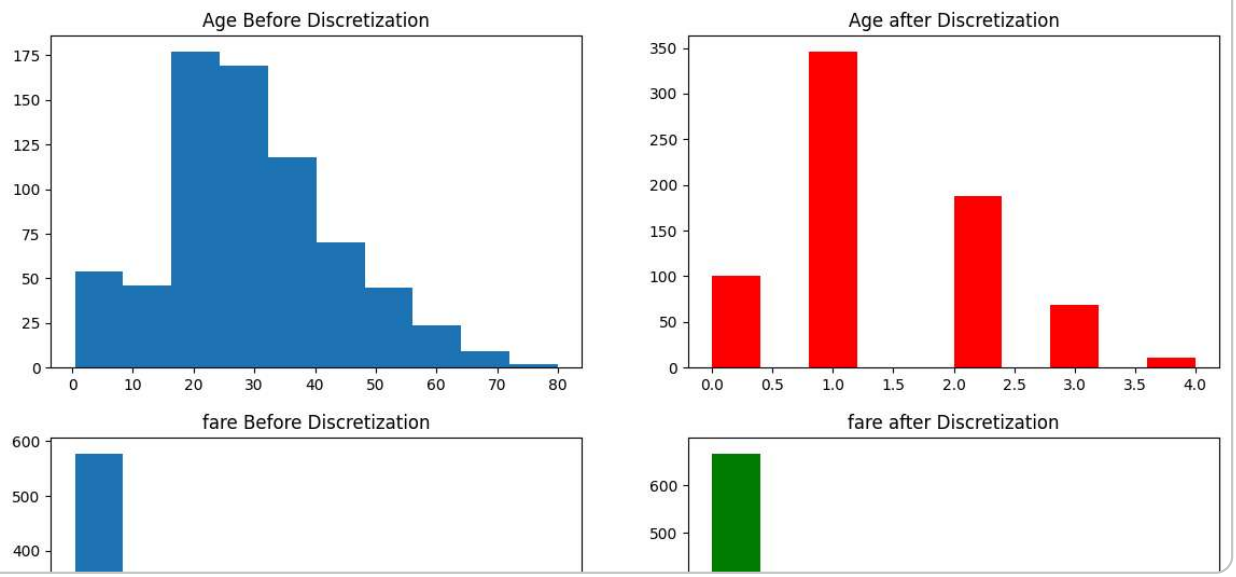
```
Test Accuracy: 0.6433566433566433
Cross-validation Accuracy: 0.67374175120654
```




```
def discretized(bins, strategy, X_train, X_test, Y_train, Y_test):
    # Create discretizers
    kb_age = KBinsDiscretizer(n_bins=bins, encode="ordinal", strategy=strategy)
```

```
    kb_fire = KBinsDiscretizer(n_bins=bins, encode="ordinal", strategy=strategy

    # ColumnTransformer
    trf = ColumnTransformer([
        ("age_bin", kb_age, [0]),
        ("fire_bin", kb_fire, [1])
    ])

    # Transform training and testing sets
    X_train_trf = trf.fit_transform(X_train)
    X_test_trf = trf.transform(X_test)

    # Train classifier
    clf = DecisionTreeClassifier()
    clf.fit(X_train_trf, Y_train)

    # Predict
    y_pred2 = clf.predict(X_test_trf)
    test_acc = accuracy_score(Y_test, y_pred2)
    print("Test Accuracy:", test_acc)

    # Cross-validation on full transformed data
    X_trf_full = trf.fit_transform(np.vstack((X_train, X_test)))
    Y_full = np.hstack((Y_train, Y_test))
    cv_acc = np.mean(cross_val_score(DecisionTreeClassifier(), X_trf_full, Y_fu
    print("Cross-validation Accuracy:", cv_acc)


    plt.figure(figsize=(14,4))
    plt.subplot(121)
    plt.hist(X["age"])
    plt.title("Age Before Discretization")

    plt.subplot(122)
    plt.hist( X_trf_full[:,0],color = "red")
    plt.title("Age after Discretization")


    plt.figure(figsize=(14,4))
    plt.subplot(121)
    plt.hist(X["fare"])
    plt.title("fare Before Discretization")

    plt.subplot(122)
    plt.hist( X_trf_full[:,1],color = "g")
    plt.title("fare after Discretization")

    return clf, test_acc, cv_acc
```

```
clf, test_acc, cv_acc = discretized(5, "uniform", X_train, X_test, Y_train, Y_t
Test Accuracy: 0.6538461538461539
```




```
clf, test_acc, cv_acc = discretized(10, "kmeans", X_t
```

```
Test Accuracy: 0.6328671328671329
Cross-validation Accuracy: 0.6499162809021963
```